

UTS
PENGOLAHAN CITRA



NAMA : Muhammad Hafizh Wijdan

NIM : 202431005

KELAS : A

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 30

ASISTEN : 1. Sharira Maharani

2. Muhammad Hanif Nuruddin Jabbar

3. Fhazel Kesra Arivi

4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	2
BAB I	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah.....	3
1.3 Manfaat Masalah.....	3
BAB II.....	5
LANDASAN TEORI	5
BAB III.....	12
HASIL	12
BAB IV	25
PENUTUP	25
DAFTAR PUSTAKA	26

BAB I PENDAHULUAN

1.1 Rumusan Masalah

- A. Menjelaskan apa yang dimaksud dengan pengolahan citra, bagaimana Pengolahan citra dapat meningkatkan kualitas dan memanipulasi dari sebuah citra digital.
- B. Bagaimana sebuah Citra Digital dapat dimanipulasi dengan mendeteksi warna melalui metode clipping.
- C. Bagaimana Komputer dapat mengenali warna dan memanipulasi citra dengan perhitungan dari sebuah aras
- D. Bagaimana Komputer dapat memperbaiki suatu citra dengan memperbaiki kontras dengan meningkatkan nilai dan menormalisasikan nilai piksel dari suatu citra

1.2 Tujuan Masalah

- A. Untuk menganalisa bagaiman komputer dapat memanipulasi dan mendekteksi suatu warn RGB dalam suatu citra digital.
- B. Untuk Menganalisa suatu komputer dapat meningkatkan kualitas gambar dengan menyebarkan nilai piksel dalam suatu gambar untuk melakukan brightening pada suatu citra digital dalm metode grayscale.
- C. Mengenali Nilai ambang batas untuk mengenali nilai Red Green Blue dalam suatu sistem komputer
- D. Mengimplenetasikan Bahasa Python dalam perbaikan dan pengolahan citra digital serta mengenali karakteristik nilai dalam bentuk histogram pada citra digital.

1.3 Manfaat Masalah

1. Bagi Penulis (Mahasiswa):

- Meningkatkan pemahaman Mahasiswa mengenai implementasi algoritma perbaikan citra menggunakan bahasa pemrograman Python.
- Mengasah kemampuan dalam menganalisis data nilai visual dengan histogram dan menentukan nilai ambang batas (thresholding dan Cliiping).
- Melatih logika pemecahan masalah dalam menghadapi kendala teknis fotografi seperti efek backlight atau kabur dan ketidak kontras suatu citra digital.

2. Bagi Pembaca/Akademisi:

- Memberikan referensi mengenai teknik konversi citra dan manipulasi kontras untuk memperbaiki kualitas visual pada objek yang membelakangi cahaya.
- Menyajikan metode ekstraksi kanal warna yang dapat digunakan sebagai dasar dalam penelitian pengenalan pola (pattern recognition) atau deteksi berdasarkan warna.

3. Secara Teknis:

- Menghasilkan prosedur kerja yang efektif untuk memisahkan informasi warna tertentu dari latar belakang yang kontras, yang dapat diaplikasikan pada digitalisasi dokumen atau tanda tangan.

BAB II

LANDASAN TEORI

SOAL 1: DETEKSI WARNA

A. Citra Digital

Citra digital adalah suatu matriks yang terdiri dari baris dan kolom dimana setiap pasang indeks baris dan kolom menyatakan suatu titik pada citra. Nilai dari setiap matriks menyatakan nilai kecerahan titik tersebut. Titik-titik tersebut dinamakan sebagai elemen citra atau piksel. Citra digital dapat didefinisikan sebagai fungsi dua variabel, $f(x,y)$, dimana x dan y adalah koordinat spasial dan nilai $f(x,y)$ adalah intensitas citra pada koordinat tersebut (T Suyono dkk, 2009).

B. Grayscale Citra grayscale,

yaitu citra yang nilai pixel-nya merepresentasikan derajat keabuan atau intensitas warna putih. Nilai intensitas paling rendah merepresentasikan warna hitam dan nilai intensitas paling tinggi merepresentasikan warna putih. Pada umumnya citra grayscale memiliki kedalaman pixel 8 bit (256 derajat keabuan), tetapi ada juga citra grayscale yang kedalaman pixel-nya bukan 8 bit, misalnya 16 bit untuk penggunaan yang memerlukan ketelitian tinggi.

C. Pengolahan Citra Digital

Pemrosesan citra digital adalah metode pemrosesan dan interpretasi citra digital berbasis persepsi visual. Data masukan dan data keluaran berupa citra merupakan fitur pengolahan dan analisis. Ungkapan "pemrosesan citra digital" mengacu pada penggunaan komputer untuk memanipulasi gambar dua dimensi. Citra digital biasanya direpresentasikan sebagai matriks 2 dimensi (2D) dengan jumlah komponen yang terbatas. Semua item dalam matriks data gambar harus memiliki koordinat x dan y yang spesifik dan berguna dengansendirinya. Setiap anggota matriks citra diberi nama piksel (elemen citra, elemen citra atau pel). Nilai setiap piksel f dalam koordinat x dan y digunakan untuk mewakili intensitas warna, yang dapat disimpan dalam 24 bit untuk gambar berwarna (dengan tigakomponen warna RGB: R= merah, G= hijau, dan B= biru), 8 bit untuk gambar skala abu-abu, atau 1 bit untuk gambar biner. Perangkat keras komputer akan mengolah citra dengan representasi digital, oleh karena itu data citra yang pada awalnya bersifat analog atau memilikinilai kecerahansembarang menurut sifat dasar objek, kemudian dikonversi menjadi data yang terkuantisasi atau hanya bernilai bilangan bulat tertentu saja (diskret). Standar yang biasa digunakan adalah 256 level intensitas (level), dengan enkripsi 8 digit (bit) atau byte atau kode biner. Ini bisa lebih dari 256 level untuk presisi, tetapi juga bisa lebih sedikit jika itu memadai. Memahami kuantitas dan kualitas

informasi yang diharapkan dari hasil pemrosesan, serta kebenaran dan kecukupan koneksi.[2]

D. Segmentasi Citra

Segmentasi citra adalah pemisahan objek yang satu dengan objek yang lain dalam suatu citra atau antara objek dengan latar yang terdapat dalam sebuah citra. Dengan proses segmentasi tersebut, masing-masing objek pada citra dapat diambil secara individu sehingga dapat digunakan sebagai input bagi proses lain. Ada 2 macam segmentasi, yaitu full segmentation dan partial segmentation. Full segmentation adalah pemisahan suatu object secara individu dari background dan diberi ID (label) pada tiap-tiap segmen. Partial segmentation adalah pemisahan sejumlah data dari background dimana data yang disimpan hanya data yang dipisahkan saja untuk mempercepat proses selanjutnya.

E. Segmentasi dengan Metode Thresholding Menurut Sutoyo (2009)

Metode thresholding sederhana termasuk thresholding global dan thresholding lokal. 46 Dalam thresholding global, satu nilai ambang digunakan untuk seluruh citra. Ini cocok untuk citra dengan latar belakang homogen. Dalam thresholding lokal, nilai ambang dihitung secara adaptif untuk setiap piksel berdasarkan lingkungan sekitarnya. Ini berguna untuk citra dengan perubahan intensitas yang signifikan. Algoritma Thresholding Terkenal Beberapa algoritma thresholding terkenal termasuk algoritma Otsu, algoritma Kapur, dan algoritma adaptif Sauvola. Algoritma Otsu mencoba untuk menentukan nilai ambang yang memaksimalkan varian antara kelas objek dan latar belakang. Algoritma Kapur menggunakan pendekatan entropi untuk menentukan nilai ambang yang optimal.

Metode ini menggunakan nilai ambang T sebagai patokan untuk memutuskan sebuah pixel diubah menjadi hitam atau putih. Biasanya dihitung dengan persamaan :

$$T = (f_{maks} + f_{min}) / 2$$

Dimana f_{maks} adalah nilai intensitas maksimum pada citra dan f_{min} adalah intensitas minimum pada citra. Jika $f(x,y)$ adalah nilai intensitas pixel pada posisi (x,y) maka pixel tersebut diganti putih atau hitam tergantung kondisi berikut.

$$g(x,y) = \begin{cases} 255 & \text{jika } f(x,y) \geq T \\ 0 & \text{jika } f(x,y) < T \end{cases}$$

METODE

1. Akuisisi Gambar melalui kamera

Penelitian ini menggunakan citra digital dengan model warna RGB (Red, Green, Blue), dan berekstensi JPG (Joint Photographic Experts Group) yang di dalamnya terdapat noise.

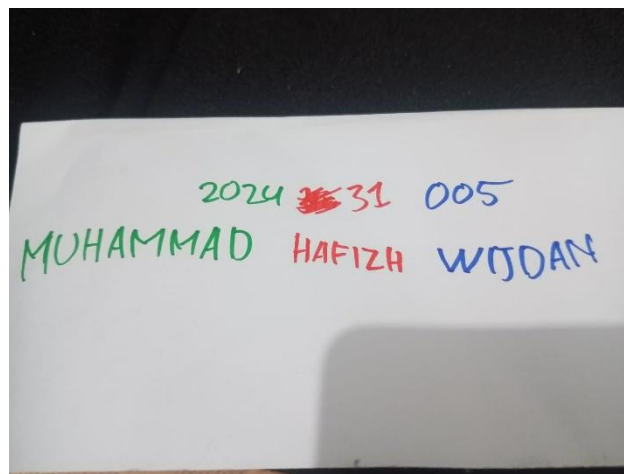
Tahap awal dalam penelitian ini adalah akuisisi citra. Akuisisi citra berfungsi menentukan data yang dibutuhkan dan metode perekaman yang sesuai [13]. Gambar dipilih menggunakan teknik purposive sampling, mencakup kondisi pencahayaan terang, redup,

dan tidak merata. Citra-citra tersebut memiliki resolusi antara 720×1080 hingga 1920×1080 piksel, sehingga dapat mewakili berbagai kondisi pencahayaan dalam fotografi digital.

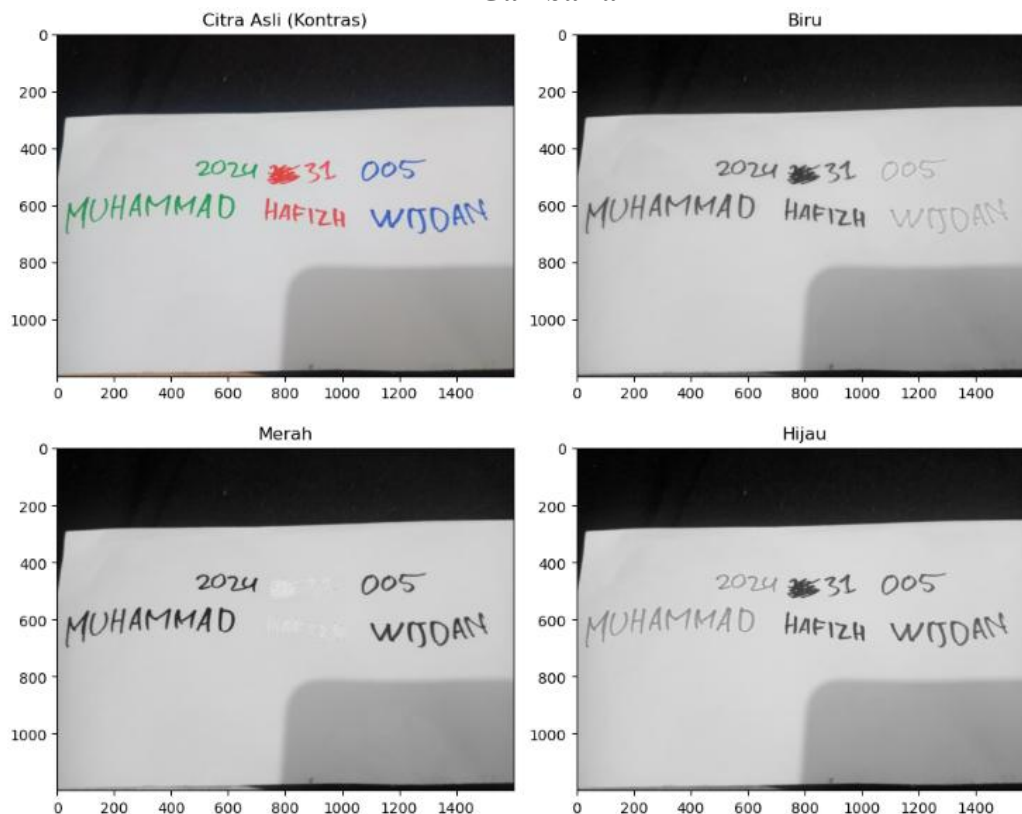
Gambar 2a menunjukkan citra input yang digunakan pada penelitian ini dengan model warna RGB. Kemudian gambar 2b adalah citra yang sudah dirubah menjadi grayscale yang kemudian akan dilakukan tahapan preprocessing.

2. Preprocessing

Pada tahapan ini citra RGB di konversi ke dalam model warna citra grayscale. Sehingga hanya memiliki satu nilai keabuan dalam tiap piksel. Proses perubahan citra RGB ke grayscale. Dengan cmap dan python pada setiap channel warna



Gambar a



Gambar b

3. Implementasi Metode Threshold

Thresholding adalah teknik OpenCV yang menentukan nilai piksel relatif terhadap ambang batas tertentu. Thresholding membandingkan setiap nilai piksel dengan ambang batasnya. Jika nilai piksel kurang dari ambang batas, maka disetel ke 0, jika tidak maka disetel ke nilai maksimum (biasanya 255). Thresholding adalah teknik segmentasi yang sangat umum yang memisahkan objek yang dianggap latar depan dari latar belakang. Ambang batas memiliki dua rentang di kedua sisinya: nilai di bawah dan di atas ambang batas. Dalam visi komputer, teknik ambang batas ini diterapkan pada gambar skala abu-abu. (rishabhsingh1304, 2023)

Analisa Histogram

Disini saya menggunakan tabel diagram untuk melihat intensitas dari satu channel warna dan menemukan min max intensitas menggunakan histogram

atau menggunakan beberapa fungsi seperti `np.min()` dan `np.max()` juga `thresh_otsu`

4. Clipping

Selanjutnya adalah metode clipping dan penggabungan dimana dari nilai ambang batas yang ditentukan dari metode manual adalah dengan $g(x,y) = \frac{f_{min}(x,y) + f_{max}(x,y)}{2}$ yang artinya mencari nilai tengah rata-rata dari sebuah intensitas dan akan dimasukkan kedalam fungsi berikut sebagai syarat manipulasi perubahan warna yang akan dirubah ke putih atau hitam

$$g(x,y) = \begin{cases} 255 & \text{jika } f(x,y) \geq T \\ 0 & \text{jika } f(x,y) < T \end{cases}$$

SOAL 2: OPERASI PIXEL

Peningkatan kontras dan kejelasan adalah salah satu tahap penting dalam pengolahan citra digital yang bertujuan untuk meningkatkan perbedaan antara tingkat kecerahan piksel dalam citra. Hal ini membantu memperjelas detail dan objek dalam citra, sehingga hasilnya lebih informatif dan dapat digunakan dalam berbagai aplikasi. Berikut adalah beberapa teknik dan konsep utama dalam peningkatan kontras dan kejelasan citra: Peningkatan Kontras

- **Histogram Equalization:** Teknik ini meratakan distribusi intensitas piksel dalam citra sehingga seluruh rentang intensitas digunakan secara optimal. Ini efektif untuk meningkatkan kontras secara umum dalam citra.
- **Kontrast Stretching:** Metode ini memperluas rentang intensitas dalam citra sehingga citra memiliki rentang kecerahan yang lebih besar. Ini berguna dalam situasi di mana citra memiliki kontras rendah.
- **Enhancement dengan Operasi Titik:** Operasi titik digunakan untuk mengubah intensitas piksel secara khusus dalam citra, seperti meningkatkan kecerahan objek tertentu.

1. Citra Digital

Representasi citra digital adalah cara untuk menggambarkan citra dalam bentuk yang dapat dipahami oleh komputer. Ini melibatkan penggunaan matriks atau array angka untuk mewakili tingkat kecerahan atau warna setiap piksel dalam citra. Dalam pengolahan citra digital, ada beberapa jenis representasi citra yang penting untuk dipahami:

- a) Citra Grayscale Citra grayscale adalah citra yang hanya memiliki tingkat kecerahan sebagai informasi. Setiap piksel dalam citra grayscale direpresentasikan oleh satu angka, biasanya dalam rentang 0 hingga 255. Nilai 0 mewakili warna hitam, sementara nilai 255 mewakili warna putih.

Untuk melakukan proses peningkatan kualitas citra, citra original diubah ke dalam citra grayscale. Grayscale yang digunakan menggunakan pembobotan nilai luminance RGB sebagaimana persamaan

$$Gray = 0.299R + 0.587G + 0.114B$$

Atau

$$Gray = 0.2126R + 0.7152G + 0.0722B$$

- b) Citra Berwarna (Mode RGB) Citra berwarna mengandung informasi kecerahan dan warna. Representasi paling umum untuk citra berwarna adalah mode RGB (Merah, Hijau, Biru). Setiap piksel dalam citra RGB direpresentasikan oleh tiga nilai, masing-masing mewakili tingkat merah, hijau, dan biru dalam rentang 0 hingga 255.
- c) Citra Biner Citra biner hanya memiliki dua nilai piksel yang mungkin: hitam dan putih. Ini adalah hasil dari proses segmentasi, di mana piksel-piksel dikelompokkan menjadi dua kelas berdasarkan ambang tertentu. Representasi citra biner menggunakan nilai biner, sering kali 0 untuk hitam dan 1 untuk putih.
- d) Matriks Citra Citra digital dapat direpresentasikan sebagai matriks dua dimensi di mana setiap elemen matriks mewakili nilai piksel di lokasi yang sesuai dalam citra. Matriks ini dapat diolah menggunakan operasi matematika untuk menghasilkan efek tertentu pada citra..

2. Kualitas Citra

Kualitas citra merujuk pada sejauh mana citra yang diambil mencerminkan objek atau scene asli dengan benar dan detail yang baik. Kualitas citra adalah faktor kunci dalam pengolahan citra digital karena hasil akhirnya sangat bergantung pada kualitas citra yang digunakan. Berikut adalah beberapa aspek penting yang memengaruhi kualitas citra:

- a) Resolusi Resolusi citra mengacu pada jumlah piksel dalam citra. Citra dengan resolusi tinggi memiliki lebih banyak detail dan lebih baik dalam merepresentasikan objek atau scene yang kompleks.
- b) Kedalaman Warna Kedalaman warna mengacu pada jumlah warna yang dapat direpresentasikan dalam citra. Citra dengan kedalaman warna yang tinggi dapat menangkap detail warna yang halus.

- c) Kecerahan dan Kontras Kecerahan mengacu pada tingkat pencahayaan dalam citra, sementara kontras mengacu pada perbedaan antara tingkat kecerahan yang berbeda dalam citra. Citra yang terlalu terang atau terlalu gelap dapat mengurangi kualitasnya.

3. Peningkatan Kualitas Citra

Perbaikan kualitas diperlukan karena seringkali citra yang dijadikan objek pembahasan mempunyai kualitas yang buruk, misalnya citra mengalami derau (noise) pada saat pengiriman melalui saluran transmisi, citra terlalu terang/gelap, citra kurang tajam, kabur, dan sebagainya. Pada proses ini, ciri-ciri tertentu yang terdapat di dalam citra lebih diperjelas kemunculannya. Secara matematis, peningkatan kualitas citra dapat diartikan sebagai proses mengubah citra $f(x, y)$ menjadi $f'(x, y)$ sehingga ciri-ciri yang dilihat pada $f(x, y)$ lebih ditonjolkan. Proses-proses yang termasuk ke dalam perbaikan kualitas citra sebagai berikut:

- a) Pengubahan kecerahan gambar (image brightness) Untuk membuat citra lebih terang atau lebih gelap, kita melakukan pengubahan kecerahan gambar. Kecerahan/kecemerlangan gambar dapat diperbaiki dengan menambahkan (atau mengurangi) sebuah konstanta kepada (atau dari) setiap pixel di dalam citra. Akibat dari operasi ini, histogram citra mengalami pergeseran. Secara matematis operasi ini ditulis sebagai $f'(x, y) = f(x, y) + b$. Jika b positif, kecerahan gambar bertambah, sebaliknya jika b negatif kecerahan gambar berkurang.
- b) Peregangan kontras (contrast stretching) Kontras menyatakan sebaran terang (lightness) dan gelap (darkness) di dalam sebuah gambar. Citra dapat dikelompokkan ke dalam tiga kategori kontras: citra kontras-rendah (low contrast), citra kontras-bagus (good contrast atau normal contrast), dan citra kontras-tinggi (high contrast). Untuk memperoleh histogram citra sesuai dengan keinginan kita, maka penyebaran nilai-nilai intensitas pada citra harus diubah. Terdapat dua metode pengubahan citra berdasarkan histogram:
 - Perataan historam (histogram equalization) Nilai-nilai intensitas di dalam citra diubah sehingga penyebarannya seragam (uniform).
 - Spesifikasi histogram (histogram specification) Nilai-nilai intensitas di dalam citra diubah agar diperoleh histogram dengan bentuk yang dispesifikasikan oleh pengguna.

Metode

1. Akuisisi Citra

Pada tahap ini, saya Tangkap atau potret citra menggunakan kamera pribadi bawaan dari handphone bahan uji coba dalam mengambil sample foto backlight atau membelakangi cahaya matahari sehingga terkesan menutupi dengan resolusi 12 MP (4032 x 3024 piksel)

2. Input Citra ke dalam komputer

Pada tahap pemrosesan awal, citra dimasukkan sebagai input ke dalam proses aplikasi aatu program yang menggunakan jupyter atau python sebagai bahasanya dan menggunakan library numpy juga computer vision

3. **Gray-Scaling (Keabuan)**

Citra asli yang berwarna (RGB) dikonversi menjadi citra *grayscale* (derajat keabuan). Hal ini dilakukan untuk menyederhanakan proses komputasi operasi piksel, di mana setiap piksel kini hanya memiliki satu nilai intensitas (0 untuk hitam pekat hingga 255 untuk putih terang), menggunakan rumus pemrosesan *luminance* standar.

4. **Operasi Piksel: Peningkatan Kecerahan (Brightness)**

Tahap selanjutnya adalah menerapkan operasi piksel titik (*point operation*) untuk meningkatkan kecerahan area subjek yang gelap. Ini dilakukan dengan menambahkan nilai konstanta (bias) β secara merata pada setiap nilai piksel citra *grayscale*. Nilai β ditentukan secara empiris dengan batasan agar area latar belakang (langit) tidak mengalami *clipping* atau melewati nilai maksimum 255.

5. **Operasi Piksel: Penyesuaian Kontras (Contrast)**

Penambahan kecerahan seringkali menyebabkan citra terlihat pudar dan kehilangan kedalaman. Oleh karena itu, dilakukan penyesuaian kontras dengan mengalikan setiap nilai piksel dengan konstanta (gain) α . Nilai α yang lebih besar dari 1.0 akan meregangkan jangkauan nilai piksel, sehingga mempertegas perbedaan antara area gelap pada subjek dan area terang pada latar belakang.

6. **Pengendalian Color Burn (Opsional: Transformasi Non-Linear/Gamma)**

Sebagai alternatif jika operasi linear (langkah 4 dan 5) menyebabkan efek *color burn* (terbakar warna) yang berlebihan pada area langit, metode transformasi kurva seperti *Gamma Correction* dapat diterapkan

$$V_{out} = 255 \cdot \left(\frac{V_{in}}{255} \right)^{\gamma}$$

Metode ini membantu meningkatkan kecerahan area bayangan (subjek) secara signifikan tanpa menaikkan intensitas area terang (langit) secara proporsional.

7. **Evaluasi dan Output Citra**

Citra hasil pemrosesan ditampilkan dan dievaluasi secara visual untuk membandingkannya dengan citra asli. Evaluasi difokuskan pada peningkatan kejelasan profil wajah/tubuh pada area yang sebelumnya gelap, serta memastikan area latar belakang yang terang tidak mengalami penurunan kualitas atau *color burn* yang berlebihan sesuai dengan toleransi yang telah ditentukan. Citra akhir kemudian disimpan sebagai output.

BAB III

HASIL

SOAL 1: DETEKSI WARNA

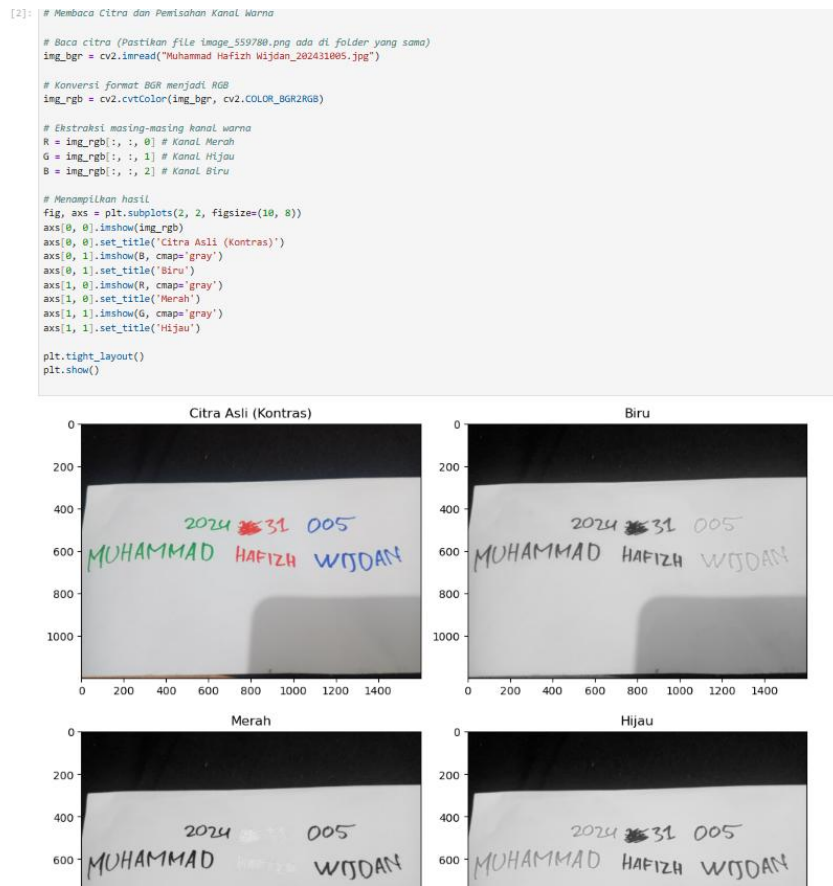
1. Deklarasi dan Import Library

Menggunakan cv2 untuk menggunakan konversi warna, thesholding dan manipulasi pixel citra dan np untuk membuat histogram dalam kebutuhan analisa

```
[1]: import cv2
import matplotlib.pyplot as plt
import numpy as np
```

2. Input Citra dan Ekstraksi Channel RGB dalam citra kedalam komputer menggunakan cmap untuk grayscale

Dalam cell ini deklarasi menggunakan imread dan ubahcv2 yang berformat bgr ke rgb lalu ambil setiap channel dengan deklarasi array yakni lebar, panjang dan channel warna maka hasilnya ia akan mendeteksi bahwa arna merah atau warna yang dituju sebagai intensitas paling mendekati



Gambar 3.1 Ekstraksi Warna

putih atau 255 sehingga tekesan memutih sedangkan warna yang tak dituju akan semakin dekat dengan 0 atau hitam pekat

3. Analisa Histogram

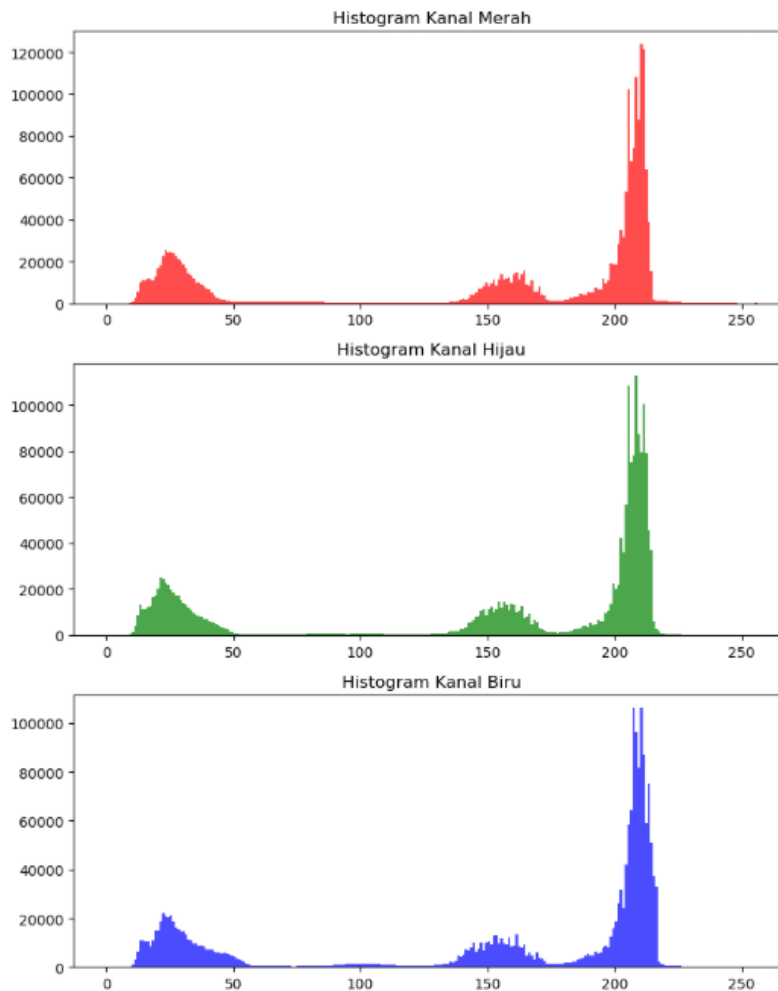
Dalam analisa histogram ini akan menggunakan ravel untuk membuat sebuah tabel diagram batang dalam melihat penyebaran intensitas warna di suatu citra menyesuaikan dengan resolusi atau panjang lebar nya

```
[3]: # Analisis Histogram

fig_hist, axs_hist = plt.subplots(3, 1, figsize=(8, 10))

axs_hist[0].hist(R.ravel(), bins=256, range=[0, 256], color='red', alpha=0.7)
axs_hist[0].set_title('Histogram Kanal Merah')
axs_hist[1].hist(G.ravel(), bins=256, range=[0, 256], color='green', alpha=0.7)
axs_hist[1].set_title('Histogram Kanal Hijau')
axs_hist[2].hist(B.ravel(), bins=256, range=[0, 256], color='blue', alpha=0.7)
axs_hist[2].set_title('Histogram Kanal Biru')

plt.tight_layout()
plt.show()
```



Gambar 3.2

```
[4]: # Analisis Histogram
fig_hist, axs_hist = plt.subplots(3, 1, figsize=(10, 12))

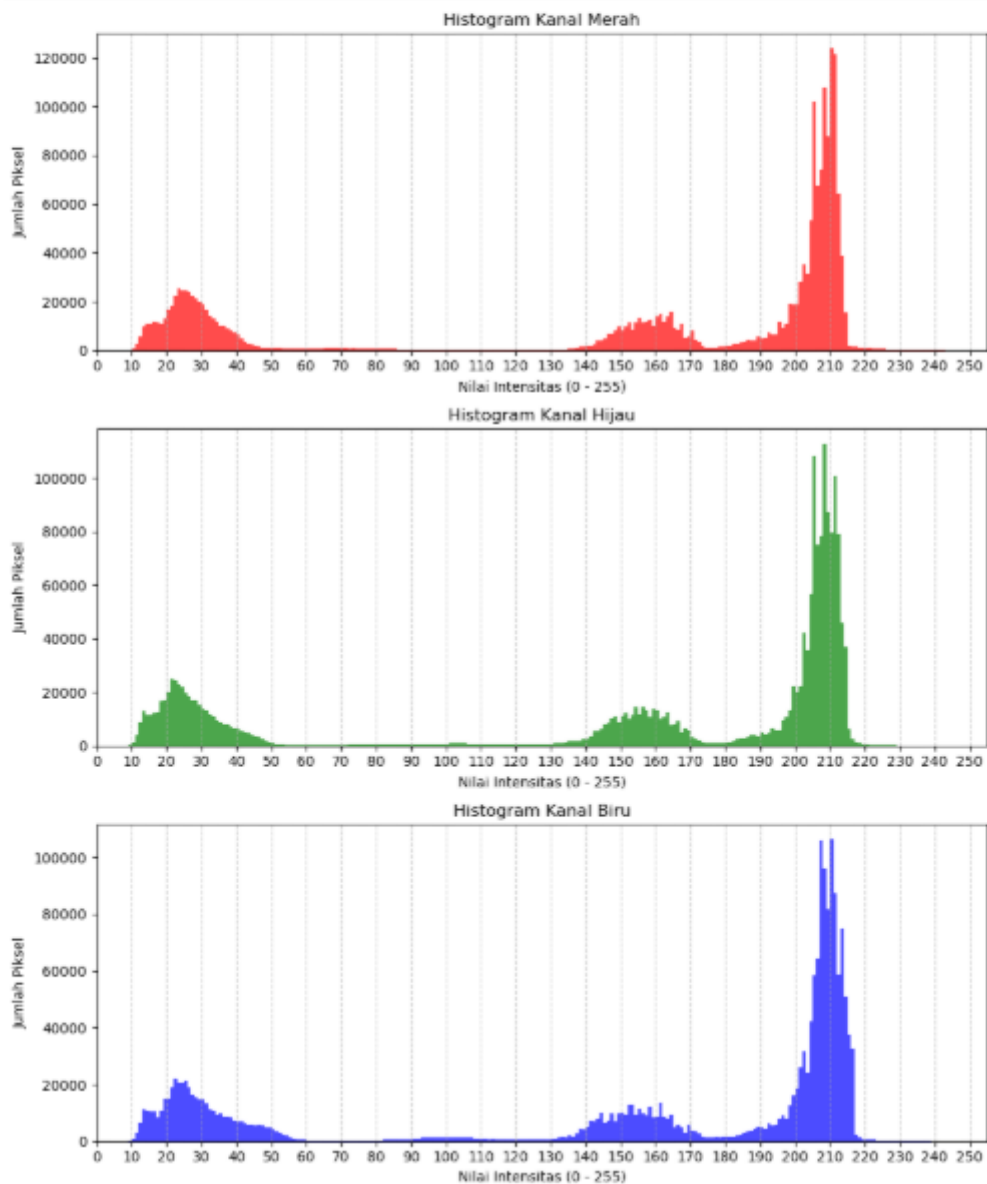
# 1. Histogram Merah
axs_hist[0].hist(R.ravel(), bins=256, range=[0, 255], color='red', alpha=0.7)
axs_hist[0].set_title('Histogram Kanal Merah')

# 2. Histogram Hijau
axs_hist[1].hist(G.ravel(), bins=256, range=[0, 255], color='green', alpha=0.7)
axs_hist[1].set_title('Histogram Kanal Hijau')

# 3. Histogram Biru
axs_hist[2].hist(B.ravel(), bins=256, range=[0, 255], color='blue', alpha=0.7)
axs_hist[2].set_title('Histogram Kanal Biru')

for ax in axs_hist:
    ax.set_xlim([0, 255])
    # angka di bawah (sumbu X) muncul setiap kelipatan 10
    ax.set_xticks(np.arange(0, 256, 10))
    # garis bantu putus-putus
    ax.grid(axis='x', linestyle='--', alpha=0.7)
    ax.set_xlabel('Nilai Intensitas (0 - 255)')
    ax.set_ylabel('Jumlah Pixel')

plt.tight_layout()
plt.show()
```



Gambar 3.3

Sebagaimana dilihat histogram menunjukkan intensitas paling banyak dimiliki oleh intensitas disekitar 10-30 dan 130-220 dengan masing masing variabel yang telah di ekstraksi, menunjukkan karakteristik

setiap warna memiliki intensitas yang berbeda beda, maka dapat ditentukan lah minimal intensitas dari satu ekstraksi minimal dan maksimal dari suatu kanal untuk membuat nilai Threshold atau ambang batas

5. Thresholding

Disini thesholding yang saya pakai adalah dengan manual pehitungan intensitas minimal dan maksimal dibagi 2 atau secara matematis $T = (f_{maks} + f_{min}) / 2$ Atau bisa menggunakan metode otsu bawaan dari cv2, disini menggunakan thesh inv untuk membalikan nilai biner 1 menjadi 0 dan 0 menjadi 1 agar teks warna kontras dengan warna kertas

```
[5]:
# Thresholding METODE OTSU (Otomatis)

# Perhitungan Otsu
ret_R_otsu, thresh_R_otsu = cv2.threshold(R, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
ret_G_otsu, thresh_G_otsu = cv2.threshold(G, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
ret_B_otsu, thresh_B_otsu = cv2.threshold(B, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

print(f"Nilai Threshold OTSU:")
print(f"- Kanal Merah : {ret_R_otsu}")
print(f"- Kanal Hijau : {ret_G_otsu}")
print(f"- Kanal Biru : {ret_B_otsu}")

# Menampilkan hasil Otsu berjejer ke samping (dalam 1 Cell)
fig_otsu, axs_otsu = plt.subplots(1, 3, figsize=(15, 5))

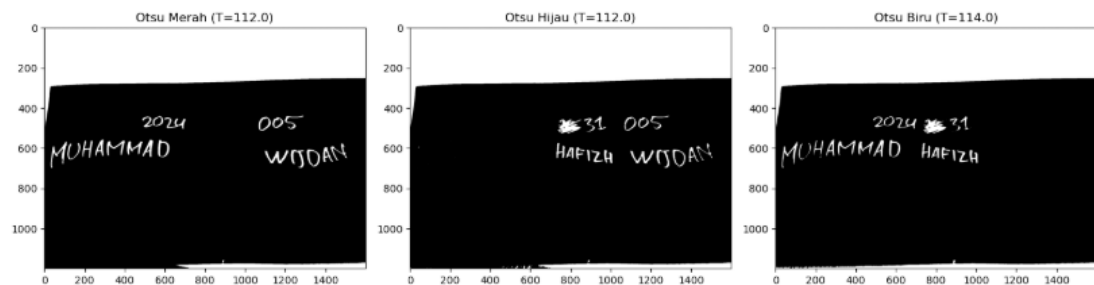
axs_otsu[0].imshow(thresh_R_otsu, cmap='gray')
axs_otsu[0].set_title(f'Otsu Merah (T={ret_R_otsu})')

axs_otsu[1].imshow(thresh_G_otsu, cmap='gray')
axs_otsu[1].set_title(f'Otsu Hijau (T={ret_G_otsu})')

axs_otsu[2].imshow(thresh_B_otsu, cmap='gray')
axs_otsu[2].set_title(f'Otsu Biru (T={ret_B_otsu})')

plt.tight_layout()
plt.show()
```

Nilai Threshold OTSU:
 - Kanal Merah : 112.0
 - Kanal Hijau : 112.0
 - Kanal Biru : 114.0



Gambar 3.4 Implementasi Threshold Otsu

Maka didapatlah nilai threshold $R = 112$, $G = 112$, dan $B = 114$.

Sedangkan apabila menggunakan manual intensitas, maka didapat sebanyak

```
[6]: #Thresholding METODE MATEMATIS (Fmin + Fmax) / 2

# 1. Mencari Fmin dan Fmax
fmin_R, fmax_R = np.min(R), np.max(R)
fmin_G, fmax_G = np.min(G), np.max(G)
fmin_B, fmax_B = np.min(B), np.max(B)

# 2. Menghitung (Fmin + Fmax) / 2
T_mid_R = int((int(fmin_R) + int(fmax_R)) / 2)
T_mid_G = int((int(fmin_G) + int(fmax_G)) / 2)
T_mid_B = int((int(fmin_B) + int(fmax_B)) / 2)

# 3. Operasi Logika Biner Manual dengan FOR LOOP dan IF-ELSE
# Kita buat matriks kosong (berisi nol/hitam) terlebih dahulu
thresh_R_mid = np.zeros_like(R, dtype=np.uint8)
thresh_G_mid = np.zeros_like(G, dtype=np.uint8)
thresh_B_mid = np.zeros_like(B, dtype=np.uint8)

# Mengambil ukuran baris (tinggi) dan kolom (lebar) gambar
baris, kolom = R.shape

print("Sedang memproses piksel dengan IF-ELSE (mungkin butuh beberapa detik)...")

# Looping mengecek setiap piksel satu per satu
# PERUBAHAN: Menyesuaikan agar hasilnya hitam putih biasa, bukan di-invers
for i in range(baris):
    for j in range(kolom):
        # Kondisi IF untuk Kanal Merah
        if R[i, j] <= T_mid_R: # Jika piksel <= Threshold (artinya piksel gelap/tinta)
            thresh_R_mid[i, j] = 0 # Jadikan HITAM (0)
        else:
            thresh_R_mid[i, j] = 255 # Jika Lebih terang dari Threshold (kertas), jadikan PUTIH (255)

        # Kondisi IF untuk Kanal Hijau
        if G[i, j] <= T_mid_G:
            thresh_G_mid[i, j] = 0
        else:
            thresh_G_mid[i, j] = 255

        # Kondisi IF untuk Kanal Biru
        if B[i, j] <= T_mid_B:
            thresh_B_mid[i, j] = 0
        else:
            thresh_B_mid[i, j] = 255

print(f"Selesai! Nilai Threshold (Fmin+Fmax)/2:")
print(f"- Kanal Merah : {T_mid_R}")
print(f"- Kanal Hijau : {T_mid_G}")
print(f"- Kanal Biru : {T_mid_B}")

# Menampilkan hasil berjejer ke samping (dalam 1 Cell)
fig_mid, axs_mid = plt.subplots(1, 3, figsize=(15, 5))

# PERUBAHAN: Pastikan cmap tetap gray, namun karena isinya dibalik, tampilannya akan putih dengan tulisan hitam/abu
axs_mid[0].imshow(thresh_R_mid, cmap='gray')
axs_mid[0].set_title(f'Manual Merah (T={T_mid_R})')

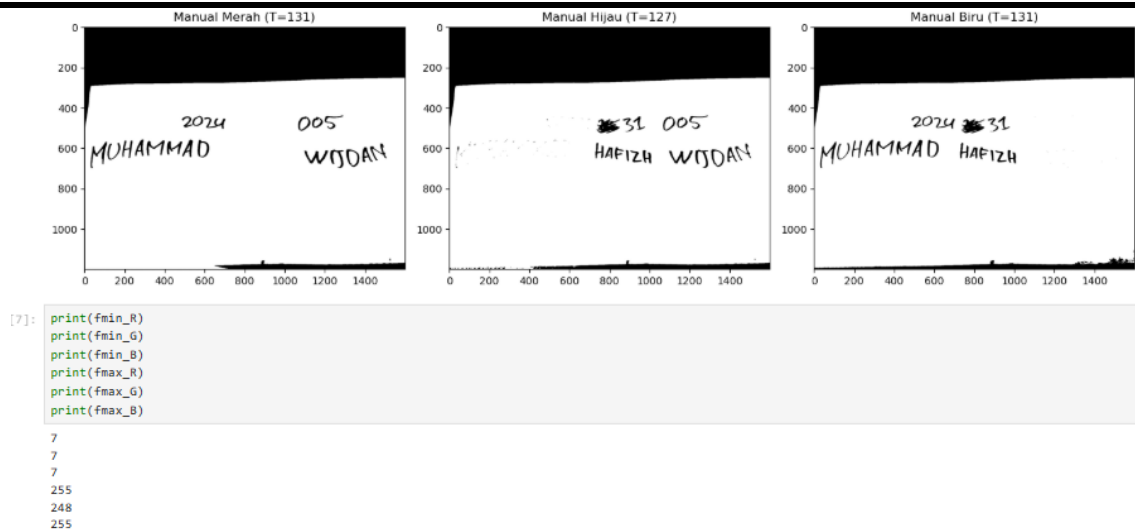
axs_mid[1].imshow(thresh_G_mid, cmap='gray')
axs_mid[1].set_title(f'Manual Hijau (T={T_mid_G})')

axs_mid[2].imshow(thresh_B_mid, cmap='gray')
axs_mid[2].set_title(f'Manual Biru (T={T_mid_B})')

plt.tight_layout()
plt.show()

Sedang memproses piksel dengan IF-ELSE (mungkin butuh beberapa detik)...
Selesai! Nilai Threshold (Fmin+Fmax)/2:
- Kanal Merah : 131
- Kanal Hijau : 127
- Kanal Biru : 131
```

Gambar 3.5 Perhitungan Threshold manual



Gambar 3.6 clipping putih threshold manual

Maka dari np.min, dan max setiap cahnnel akan didapatlah hasil Thresh dari R 127, G 135 dan 128 hasil dari perhitunga fmin dan fmax yang telah didapat dengan

$$T = \text{Min} + \text{Max} / 2$$

Maka masing masing adalah

$$TR = (7 + 255 / 2) = 131$$

$$TG = (7 + 248 / 2) = 127$$

$$TB = (7 + 255) / 2 = 131$$

6. Implementasi Clipping

Dalama menerapkan konsep clipping yakni menntukan pixel mana yang akan dideteksi sebagai waran yang dituju dapat menggunakan penggunaan if juga meng inverse kan dari prthitungan threshold tadi

Juga menggunakan maipulasi biwise dan or sehingga menggabungkan atau memisaknakan gambar, dibawah adalah penggunaan bitwise

```
[8]: #PENGGABUNGAN (CLIPPING / OPERASI LOGIKA)

# Invers (NOT) hasil thresholding manual agar Latar jadi hitam, tulisan jadi putih
inv_R = cv2.bitwise_not(thresh_R_mid)
inv_G = cv2.bitwise_not(thresh_G_mid)
inv_B = cv2.bitwise_not(thresh_B_mid)

# Membuat kanvas kosong dengan ukuran yang sama (untuk NONE)
none_canvas = np.zeros_like(inv_R)

# Logika Penggabungan Warna (Menggunakan gambar yang sudah di-Invers)

# 1. BLUE: Warna Biru ada di tulisan yang hilang/pudar di kanal Merah dan kanal Hijau.
blue_only = cv2.bitwise_and(inv_R, inv_G)

# 2. RED-BLUE: Warna Merah dan Biru hilang/pudar di kanal Hijau.
red_blue = inv_G

# 3. RED-GREEN-BLUE: Semua warna.
red_green_blue = cv2.bitwise_or(inv_R, inv_B)

# Menampilkan hasil (seperti Poin F di soal)
fig_clip, axs_clip = plt.subplots(2, 2, figsize=(12, 8))

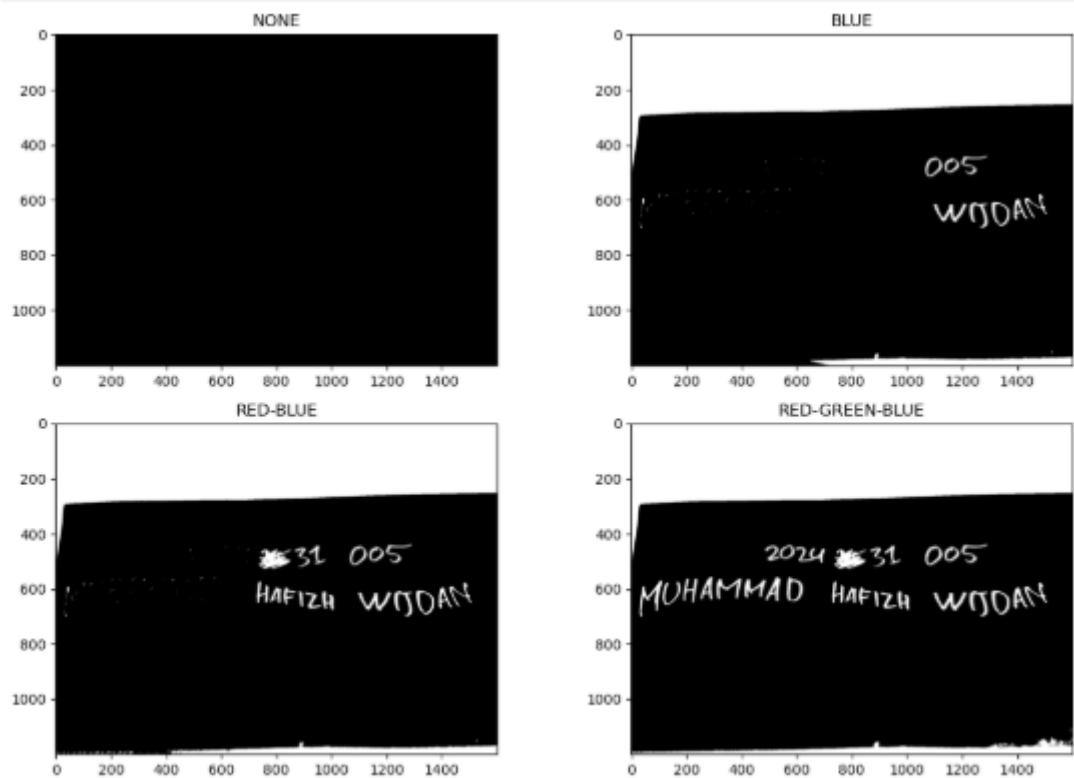
axs_clip[0, 0].imshow(none_canvas, cmap='gray')
axs_clip[0, 0].set_title('NONE')

axs_clip[0, 1].imshow(blue_only, cmap='gray')
axs_clip[0, 1].set_title('BLUE')

axs_clip[1, 0].imshow(red_blue, cmap='gray')
axs_clip[1, 0].set_title('RED-BLUE')

axs_clip[1, 1].imshow(red_green_blue, cmap='gray')
axs_clip[1, 1].set_title('RED-GREEN-BLUE')

plt.tight_layout()
plt.show()
```



Gambar 3.7 clipping inverse otsu

Sedangkan penggunaan if else untuk membalikan nilai putih dan hitam sebagai berikut dan hasilnya

```
[9]: none_canvas = np.zeros_like(R, dtype=np.uint8)
blue_only = np.zeros_like(R, dtype=np.uint8)
red_blue = np.zeros_like(R, dtype=np.uint8)
red_green_blue = np.zeros_like(R, dtype=np.uint8)

print("Sedang memproses Logika Clipping dengan IF-ELSE (mungkin butuh beberapa detik)...")

# Looping mengecek setiap piksel dari hasil Thresholding SEL 4
for i in range(baris):
    for j in range(kolom):
        # 1. BLUE: Warna Biru bernilai Hitam (0) di kanal Merah DAN kanal Hijau.
        # Logika AND: Jika keduanya 0, jadikan Putih (255)
        if thresh_R_mid[i, j] == 0 and thresh_G_mid[i, j] == 0:
            blue_only[i, j] = 255

        # 2. RED-BLUE: Tinta Merah dan Biru bernilai Hitam (0) di kanal Hijau.
        if thresh_G_mid[i, j] == 0:
            red_blue[i, j] = 255

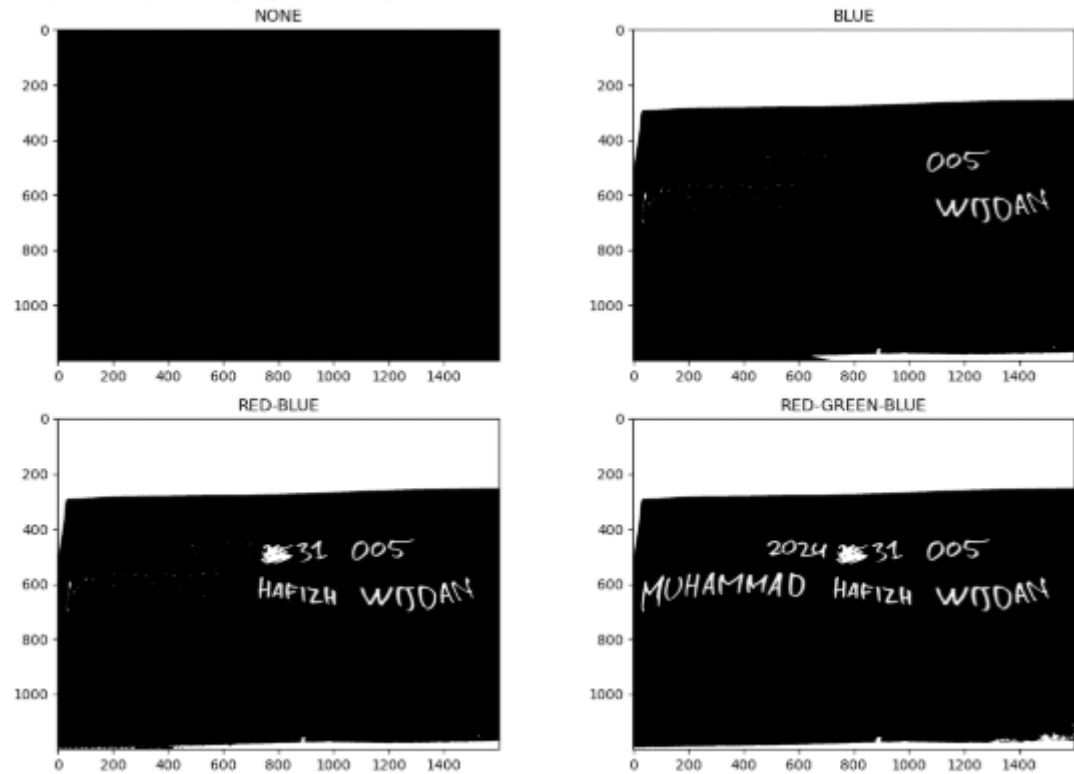
        # 3. RED-GREEN-BLUE: Semua warna.
        # Logika OR: Jika gelap di kanal Merah (dapat Hijau, Biru) ATAU gelap di kanal Biru (dapat Merah, Hijau)
        if thresh_R_mid[i, j] == 0 or thresh_B_mid[i, j] == 0:
            red_green_blue[i, j] = 255

# Menampilkan hasil (seperti Poin F di soal)
fig_clip, axs_clip = plt.subplots(2, 2, figsize=(12, 8))

axs_clip[0, 0].imshow(none_canvas, cmap='gray'); axs_clip[0, 0].set_title('NONE')
axs_clip[0, 1].imshow(blue_only, cmap='gray'); axs_clip[0, 1].set_title('BLUE')
axs_clip[1, 0].imshow(red_blue, cmap='gray'); axs_clip[1, 0].set_title('RED-BLUE')
axs_clip[1, 1].imshow(red_green_blue, cmap='gray'); axs_clip[1, 1].set_title('RED-GREEN-BLUE')

plt.tight_layout()
plt.show()
```

Sedang memproses Logika Clipping dengan IF-ELSE (mungkin butuh beberapa detik)...



Gambar 3.8 clipping inverse manual threshold

SOAL 2: OPERASI PIXEL

1. Import Library

`cv2` (OpenCV): Digunakan untuk membaca matriks gambar dari direktori komputer.
`numpy` (sebagai `np`): Digunakan untuk manipulasi array dan matriks secara efisien, serta mengatur tipe data matematis pada piksel gambar.
`matplotlib.pyplot` (sebagai `plt`): Digunakan untuk melakukan *plotting*, menampilkan gambar, dan membuat grafik histogram.

```
OPERASI PIXEL
[15]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

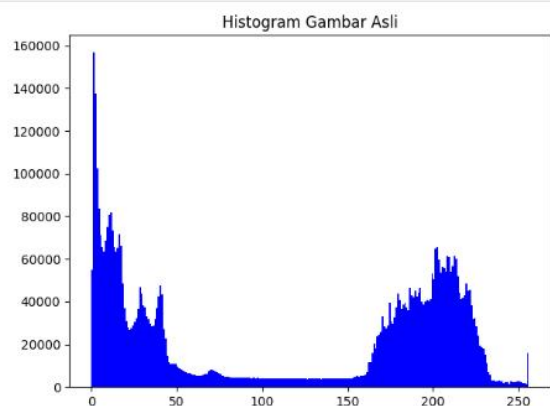
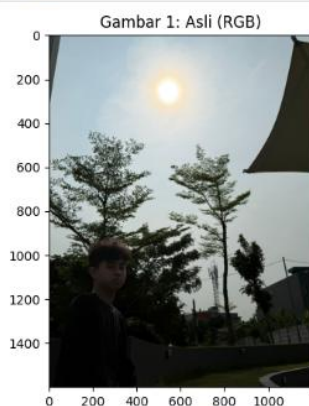
2. Membaca Data & Menampilkan Gambar 1 (Asli)

Pada tahapan ini, citra foto dengan efek backlight (latar belakang sangat terang, profil gelap) dibaca ke dalam memori menggunakan `cv2.imread()`. Secara default, OpenCV membaca gambar dalam format warna BGR (Blue-Green-Red). Agar gambar tidak terlihat kebiruan ketika ditampilkan dengan Matplotlib, dilakukan konversi ruang warna dari BGR ke RGB menggunakan perintah `cv2.cvtColor()`. Selanjutnya, dimensi gambar (jumlah baris dan kolom piksel) disimpan untuk kebutuhan proses looping di tahap berikutnya. Citra asli ditampilkan berdampingan dengan histogramnya untuk menganalisis distribusi intensitas cahaya sebelum diproses.

Membaca Data & Gambar 1 (Asli)

```
[16]: img = cv2.imread("WhatsApp Image 2026-05-14 at 14.28.53.jpg")
[baris, kolom] = img.shape[:2]
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Tampilkan Gambar 1 (Asli) dan Histogram
fig, axs = plt.subplots(1, 2, figsize=(15, 5))
axs[0].imshow(rgb)
axs[0].set_title("Gambar 1: Asli (RGB)")
axs[1].hist(rgb.ravel(), bins=256, range=[0, 256], color='blue')
axs[1].set_title("Histogram Gambar Asli")
plt.show()
```



3. Gambar 2: Konversi ke Grayscale (Keabuan)

Gambar berwarna (RGB) yang memiliki 3 channel dikonversi menjadi gambar grayscale (1 channel) secara manual melalui proses perulangan bersarang (nested loop).

Rumus yang digunakan adalah pendekatan standar luminansi televisi (NTSC):

$$f(x,y) = 0.299 * R + 0.587 * G + 0.114 * B$$

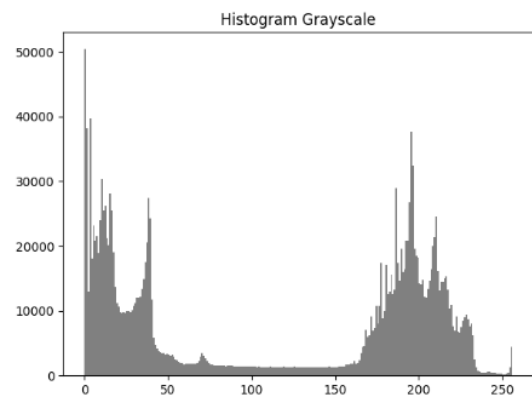
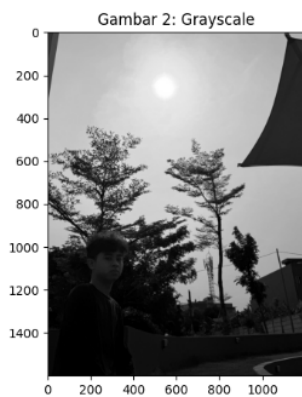
Nilai desimal (R, G, B) yang memiliki bobot berbeda tersebut ditambahkan dan diubah menjadi integer untuk merepresentasikan tingkat keabuan mata manusia. Hasil dari langkah ini menjadi citra dasar `f(x,y)` untuk diproses pada tahap kecerahan dan kontras.

Gambar 2: Konversi Grayscale

```
[17]: gray = np.zeros((baris, kolom), dtype=np.uint8)

for x in range(baris):
    for y in range(kolom):
        r = rgb[x, y, 0]
        g = rgb[x, y, 1]
        b = rgb[x, y, 2]
        # Rumus Luminance standar
        f = 0.299 * r + 0.587 * g + 0.114 * b
        gray[x, y] = int(f)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
axes[0].imshow(gray, cmap='gray')
axes[0].set_title("Gambar 2: Grayscale")
axes[1].hist(gray.ravel(), bins=256, range=[0, 256], color='gray')
axes[1].set_title("Histogram Grayscale")
plt.show()
```



4. Gambar 3: Operasi Kecerahan (Penambahan Nilai Beta)

Operasi kecerahan bertujuan untuk mengangkat intensitas cahaya, terutama pada area profil wajah/tubuh yang gelap akibat backlight. Hal ini dilakukan dengan menambahkan nilai skalar Beta ($\beta = 60$) ke setiap piksel.

Rumus yang digunakan: $g(x,y) = f(x,y) + \beta$

tipe data `uint8` (bernilai 0-255) memiliki kelemahan *integer overflow*. Apabila piksel langit (misal: 200) ditambah 60, hasilnya adalah 260. Karena kapasitas `uint8` hanya sampai 255, komputer akan meresetnya menjadi angka 4 (menjadi hitam/abu-abu gelap). *error* inilah penyebab utama *color burn* parah pada gambar.

Oleh karena itu, kode melakukan *castin* (mengubah sementara tipe data piksel ke `float` sebelum operasi matematika), lalu memotong paksa nilainya di batas maksimum 255 menggunakan perintah `np.clip()`.

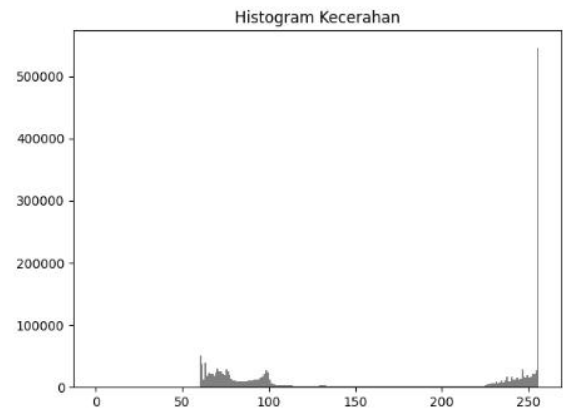
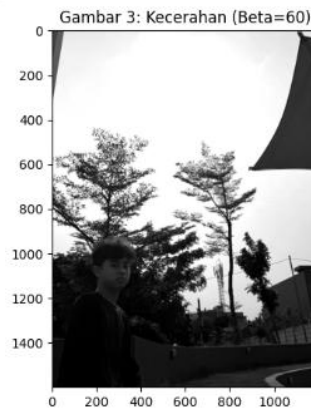
Gambar 3: Operasi Kecerahan (Beta)

```
[18]: beta = 60
citra_cerah = np.zeros((baris, kolom), dtype=np.float64)

for x in range(baris):
    for y in range(kolom):
        # PENTING: Cast ke float untuk menghindari overflow sebelum np.clip
        gx = float(gray[x, y]) + beta
        citra_cerah[x, y] = gx

citra_cerah = np.clip(citra_cerah, 0, 255).astype(np.uint8)

fig, axs = plt.subplots(1, 2, figsize=(15, 5))
axs[0].imshow(citra_cerah, cmap='gray')
axs[0].set_title(f"Gambar 3: Kecerahan (Beta={beta})")
axs[1].hist(citra_cerah.ravel(), bins=256, range=[0, 256], color='gray')
axs[1].set_title("Histogram Kecerahan")
plt.show()
```



5. Gambar 4: Operasi Kontras (Perkalian dengan Alpha)

Operasi kontras bertujuan untuk memperlebar jarak/rentang intensitas antar piksel sehingga perbedaan antara area gelap dan terang semakin tajam. Hal ini dilakukan dengan mengalikan setiap intensitas piksel dengan nilai skalar **Alpha** ($\alpha = 1.4$). Rumus yang digunakan: $g(x,y) = f(x,y) * \alpha$

Serupa dengan operasi kecerahan, setiap nilai piksel dikonversi ke `float` terlebih dahulu. Setelah dikalikan, nilai yang melebihi batas 255 akan dipangkas secara aman oleh fungsi `np.clip()` agar tidak terjadi **color burn** (terbakar warna yang mengakibatkan gambar rusak/pecah).

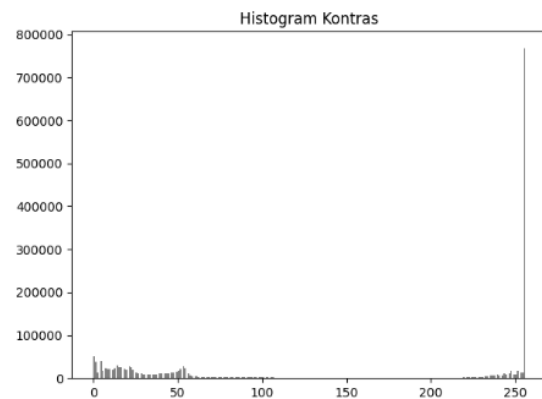
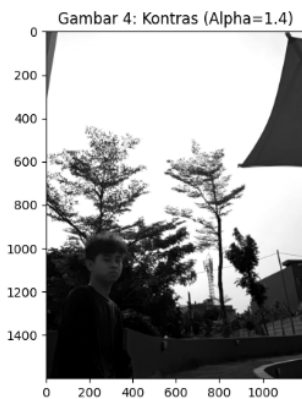
Gambar 4: Operasi Kontras (Alpha)

```
[19]: alpha = 1.4
citra_kontras = np.zeros((baris, kolom), dtype=np.float64)

for x in range(baris):
    for y in range(kolom):
        gx = float(gray[x, y]) * alpha
        citra_kontras[x, y] = gx

citra_kontras = np.clip(citra_kontras, 0, 255).astype(np.uint8)

fig, axs = plt.subplots(1, 2, figsize=(15, 5))
axs[0].imshow(citra_kontras, cmap='gray')
axs[0].set_title(f"Gambar 4: Kontras (Alpha={alpha})")
axs[1].hist(citra_kontras.ravel(), bins=256, range=[0, 256], color='gray')
axs[1].set_title("Histogram Kontras")
plt.show()
```



6. Gambar 5: Operasi Gabungan (Logika Piecewise Linear)

implementasi utama untuk skasus "meningkatkan kontras di area gelap dan mengurangi kontras di area terang". Operasi gabungan linear biasa $(\alpha * f(x,y) + \beta)$ tidak cukup karena akan membuat langit yang sudah terang dipaksa menjadi lebih putih solid, yang merusak detail langit (kualitas visual menurun).

Sebagai solusinya, diterapkan algoritma Piecewise Linear Transformation menggunakan kondisi If-Else:

1. Jika Intensitas Piksel < 128 (Area Gelap / Profil):

Di area wajah dan baju, diterapkan $\alpha > 1$ (misal 1.5) untuk meningkatkan kontras sehingga tekstur profil terlihat jelas, dan ditambahkan nilai β yang besar (misal 40) untuk mencerahkan bayangan gelapnya.

2. Jika Intensitas Piksel ≥ 128 (Area Terang / Langit Latar Belakang):

Di area latar belakang, diterapkan $\alpha < 1$ (misal 0.5) untuk mengurangi kontras. Pengurangan kontras di area ini berfungsi untuk menekan rentang piksel langit agar tidak menabrak batas 255 (mencegah color burn). Dengan penyesuaian nilai skalar ini, detail awan dan langit terang tetap terselamatkan secara wajar.

Hasil akhirnya disatukan dengan `np.clip()`

Gambar 5: Operasi Gabungan (Meningkatkan Kontras Gelap, Mengurangi Kontras Terang)

```
[20]: citra_gabungan = np.zeros((baris, kolom), dtype=np.float64)

for x in range(baris):
    for y in range(kolom):
        f = float(gray[x, y])

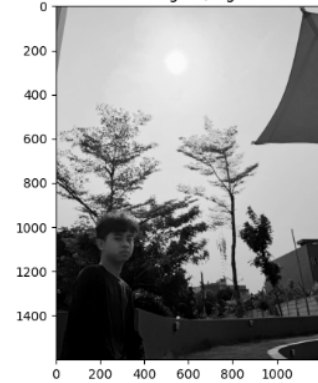
        if f < 128:
            # Area gelap (profil): Alpha besar, Beta besar
            gyx = f * 1.5 + 40
        else:
            # Area terang (Langit): Alpha kecil (kurangi kontras), Beta disesuaikan agar nyambung
            gyx = f * 0.5 + 100

        citra_gabungan[x, y] = gyx

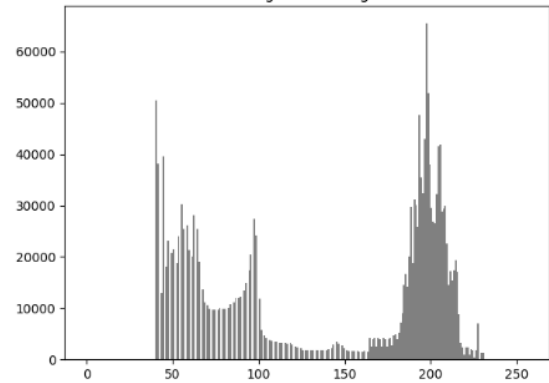
citra_gabungan = np.clip(citra_gabungan, 0, 255).astype(np.uint8)

fig, axs = plt.subplots(1, 2, figsize=(15, 5))
axs[0].imshow(citra_gabungan, cmap='gray')
axs[0].set_title("Gambar 5: Gabungan (Logika Piecewise)")
axs[1].hist(citra_gabungan.ravel(), bins=256, range=[0, 256], color='gray')
axs[1].set_title("Histogram Gabungan")
plt.show()
```

Gambar 5: Gabungan (Logika Piecewise)



Histogram Gabungan



BAB IV

PENUTUP

Berdasarkan landasan teori dan hasil pengujian praktikum pengolahan citra digital yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Konsep Dasar Manipulasi Citra:** Pengolahan citra digital pada dasarnya adalah proses memanipulasi matriks nilai piksel secara matematis. Penggunaan bahasa pemrograman Python dengan pustaka seperti OpenCV, NumPy, dan Matplotlib terbukti sangat efektif untuk melakukan konversi ruang warna, ekstraksi kanal warna, analisis, hingga perbaikan kualitas gambar.
2. **Analisis Histogram dan Thresholding:** Analisis histogram pada setiap kanal warna (Red, Green, Blue) sangat penting untuk melihat pola distribusi intensitas cahaya pada gambar. Pemahaman terhadap nilai intensitas minimum ($f_{\min}$) dan maksimum ($f_{\max}$) ini menjadi dasar penentuan nilai ambang batas (threshold), baik menggunakan perhitungan matematis manual maupun secara otomatis menggunakan metode Otsu, guna memisahkan objek tulisan dari latar belakang kertas.
3. **Deteksi Warna Melalui Operasi Logika (Clipping):** Pemisahan warna spesifik dari citra berhasil dilakukan dengan metode clipping yang menggabungkan hasil thresholding biner dengan operasi logika boolean (seperti INVERS, AND, dan OR). Hal ini memungkinkan sistem komputer untuk mendeteksi, menghilangkan, atau menonjolkan warna tinta tertentu pada gambar.
4. **Operasi Piksel untuk Peningkatan Kualitas Citra:** Peningkatan kualitas gambar yang gelap (seperti kasus backlight) dapat diatasi menggunakan operasi piksel titik (point operation). Penambahan konstanta (nilai beta) pada matriks citra berfungsi menaikkan kecerahan (brightness), sementara perkalian dengan skalar (nilai alpha) berfungsi meregangkan atau mengatur kontras citra. Dalam proses ini, penggunaan fungsi pembatasan seperti np.clip sangat krusial untuk mencegah integer overflow yang menyebabkan efek color burn (warna rusak/putih solid).
5. **Keunggulan Transformasi Linier Sepotong (Piecewise Linear Transformation):** Menerapkan perbaikan kecerahan dan kontras secara merata pada foto backlight seringkali merusak area latar belakang (langit) yang sudah terang. Pendekatan terbaik yang didapat adalah menggunakan algoritma Piecewise Linear, di mana nilai alpha dan beta dibedakan berdasarkan ambang batas intensitas area terang dan gelap. Hasilnya, area gelap pada subjek berhasil diterangkan dan diperjelas secara maksimal, sementara area terang tetap dipertahankan detailnya tanpa mengalami over-exposure.

DAFTAR PUSTAKA

- [1] Aulia, R., Gibran Airoso, M. R. F., Nabila S., S., & Budiarti, N. A. E. (2026). Peningkatan Kualitas Citra Fotografi menggunakan Metode Histogram Equalization (HE) dan Adaptive Histogram Equalization (AHE). JUSTIN (Jurnal Sistem dan Teknologi Informasi), 14(1), 46-50. <https://jurnal.untan.ac.id/index.php/justin/article/view/93381>
- [2] Azizah, A. Z. S., Yalis, F. N. T., Narayana, I. S., Syalom, K. A., & Rosyani, P. (2024). Pengolahan Citra Digital Dengan Penerapan Teknik Ambang Batas: Studi Kasus Menggunakan Opencv. Jurnal AI dan SPK: Jurnal Artificial Inteligent dan Sistem Penunjang Keputusan, 1(4), 283-287. <https://jurnalmahasiswa.com/index.php/aidanspk>
- [3] Heryanto, I. W. A., Artama, Kurniawan, M. W. S., & Gunadi, I. G. A. (2020). Segmentasi Warna Dengan Metode Thresholding. Wahana Matematika dan Sains: Jurnal Matematika, Sains, dan Pembelajarannya, 14(1), 54-64.
- [4] Prayogi, D., Lubis, S. R., Rizko, M. A., & Suteja, A. G. (2025). Optimalisasi Segmentasi Citra Digital Metode Canny Edge Detection dan Thresholding. Journal of Artificial Intelligence and Digital Business (RIGGS), 4(2), 6334-6339. <https://journal.ilmudata.co.id/index.php/RIGGS>
- [5] Purwanto, D., & Agustiyar. (2024). Global Thresholding Implementation for Noise Handling in Digital Image Recognition. Jurnal Transformatika, 21(2), 93-100. <https://journals.usm.ac.id/index.php/transformatika/>
- [6] Siagian, S., Ibnutama, K., & Mahyuni, R. (2022). Implementasi Metode Ekstraksi Ciri Warna Untuk Mendeteksi Kematangan Buah Jeruk. Jurnal Sistem Informasi TGD, 1(6), 898-905. <https://ojs.trigunadharma.ac.id/index.php/jsi>
- [7] Dijaya, R., & Setiawan, H. (2023). Buku Ajar Pengolahan Citra Digital. Sidoarjo: UMSIDA Press. ISBN: 978-623-464-075-5. Tautan: <https://press.umsida.ac.id/index.php/umsidapress/article/view/978-623-464-075-5> (atau DOI: <https://doi.org/10.21070/2023/978-623-464-075-5>)